

Diffusion models

Weekend 3, Saturday morning

Carlos Cotrini

CAS Building ML/AI Applications, ETH Zurich

Saturday 3 October 2026

Four main tasks in computer vision

Regression and
classification of images



Cat? Yes

Dog? Yes

Bat? No

Feature extraction



Cat color: black

Dog color: brown

Cat's eye color:
green

Dog's eye color:
brown

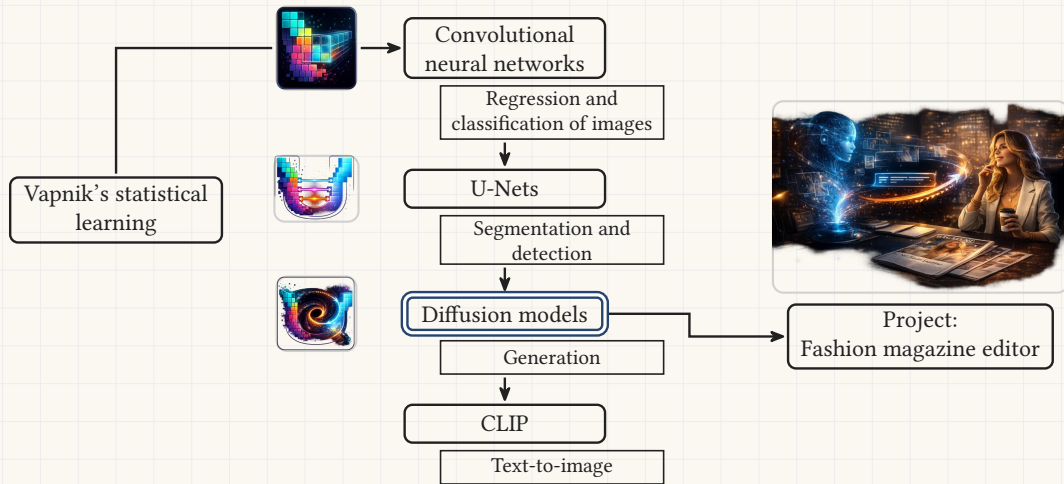
Segmentation and
detection



Generation



Agenda

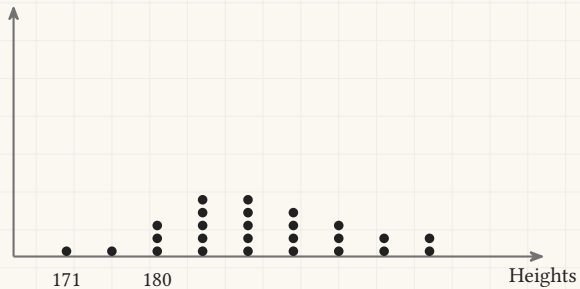


1

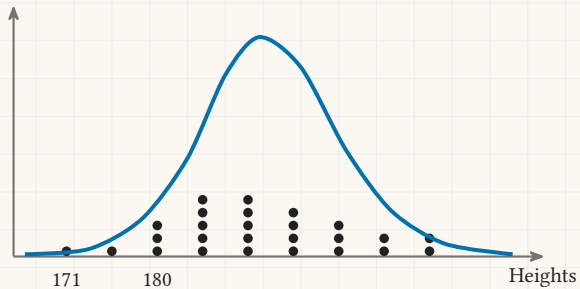
Section 1

Diffusion phenomena

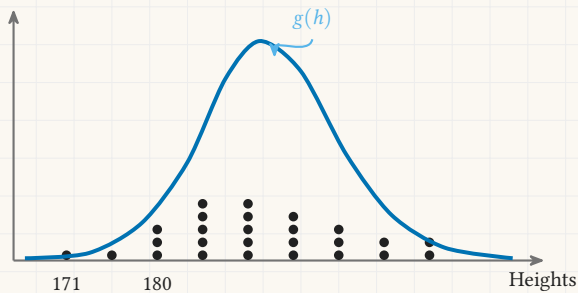
Diffusion phenomena, in one dimension



Diffusion phenomena, in one dimension

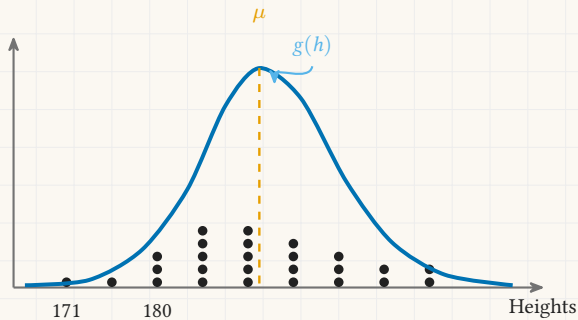


Diffusion phenomena, in one dimension



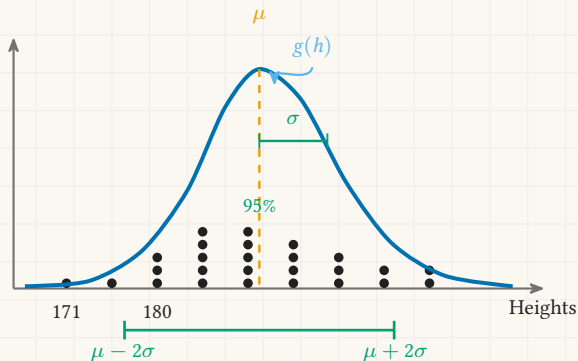
$$g(h) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(h - \mu)^2\right)$$

Diffusion phenomena, in one dimension



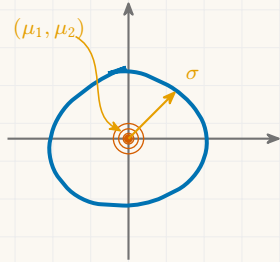
$$g(h) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(h - \mu)^2\right)$$

Diffusion phenomena, in one dimension

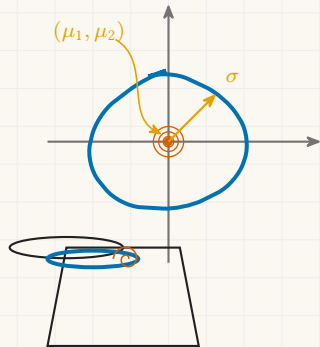


$$g(h) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(h - \mu)^2\right)$$

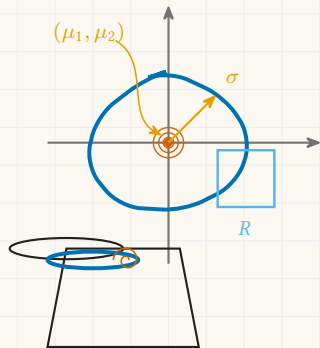
Diffusion phenomena, in two dimensions



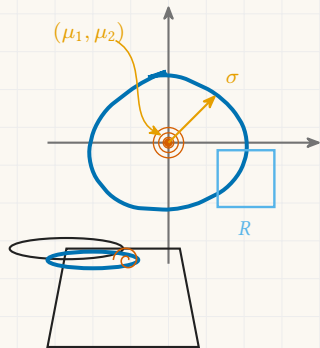
Diffusion phenomena, in two dimensions



Diffusion phenomena, in two dimensions



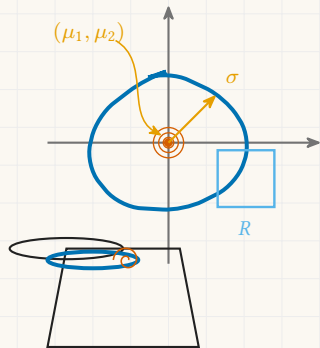
Diffusion phenomena, in two dimensions



$$= \iint_R g(x_1, x_2) dx_1 dx_2$$

Fraction of ink in R

Diffusion phenomena, in two dimensions

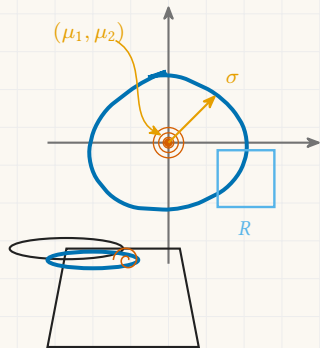


Fraction of ink in R

$$= \iint_R g(x_1, x_2) dx_1 dx_2$$

$$\approx \sum_{x_1^*, x_2^* \in \square} g(x_1^*, x_2^*) \cdot \text{Area}(\square)$$

Diffusion phenomena, in two dimensions



Fraction of ink in R

$$= \iint_R g(x_1, x_2) dx_1 dx_2$$

$$\approx \sum_{x_1^*, x_2^* \in \square} g(x_1^*, x_2^*) \cdot \text{Area}(\square)$$

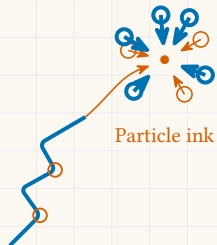
$$g(x_1, x_2) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{1}{2\sigma^2} [(x_1 - \mu_1)^2 + (x_2 - \mu_2)^2]\right)$$

Simulating diffusion

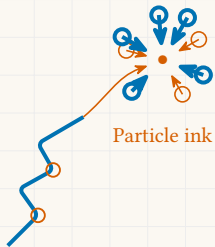


Particle ink

Simulating diffusion



Simulating diffusion

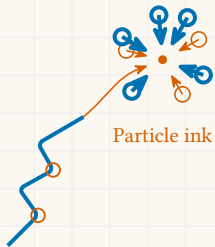


Procedure to simulate

Access to a function ρ

ρ $\rightarrow$ Random numbers in $[-1, 1]$

Simulating diffusion



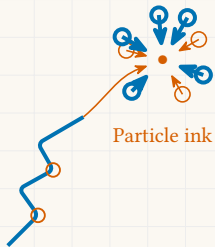
Procedure to simulate

Access to a function ρ

ρ → Random numbers in $[-1, 1]$

u_1 → -0.43

Simulating diffusion



Particle ink

Procedure to simulate

Access to a function ρ

ρ → Random numbers in $[-1, 1]$

$u_1 \rightarrow -0.43$

$\vdots$

$u_n \rightarrow 0.2$

Simulating diffusion



Particle ink

$$\mu + \sigma z =$$

Procedure to simulate

Access to a function ρ

ρ → Random numbers in $[-1, 1]$

$$u_1 \rightarrow -0.43$$

$\vdots$

$$u_n \rightarrow 0.2$$

$$u_1 + \dots + u_n \approx$$

Simulating diffusion



Particle ink

What if I want centre of mass
in μ ?

$$\mu + \sigma z =$$

Procedure to simulate

Access to a function ρ

$\rho \rightarrow$ Random numbers in $[-1, 1]$

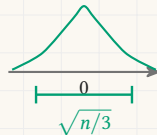
$$u_1 \rightarrow -0.43$$

$\vdots$

$$u_n \rightarrow 0.2$$

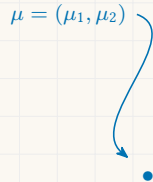
$$u_1 + \dots + u_n \approx$$

$$\sigma = 7$$



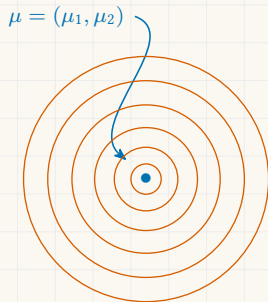
Fabricating a two-dimensional diffusion

Fabricate two-dimensional diffusions with a centre μ and a spread σ ?



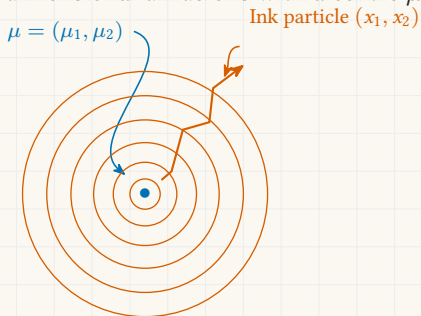
Fabricating a two-dimensional diffusion

Fabricate two-dimensional diffusions with a centre μ and a spread σ ?



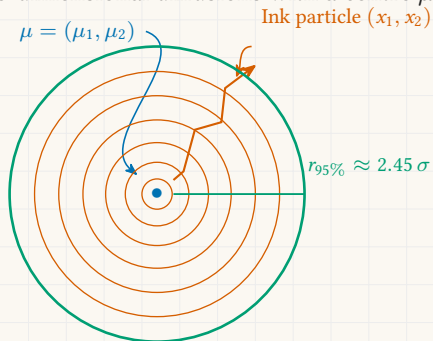
Fabricating a two-dimensional diffusion

Fabricate two-dimensional diffusions with a centre μ and a spread σ ?



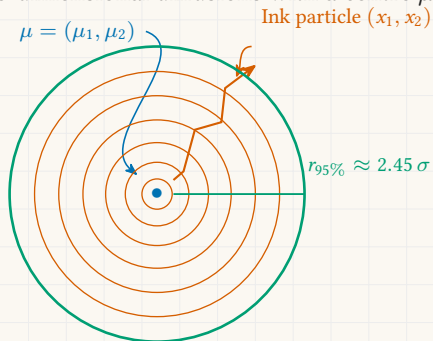
Fabricating a two-dimensional diffusion

Fabricate two-dimensional diffusions with a centre μ and a spread σ ?



Fabricating a two-dimensional diffusion

Fabricate two-dimensional diffusions with a centre μ and a spread σ ?



Run sampling twice: one for x_1 ,
another for x_2 .

2

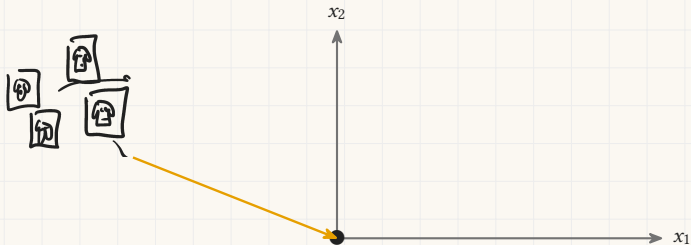
Section 2

Diffusion models

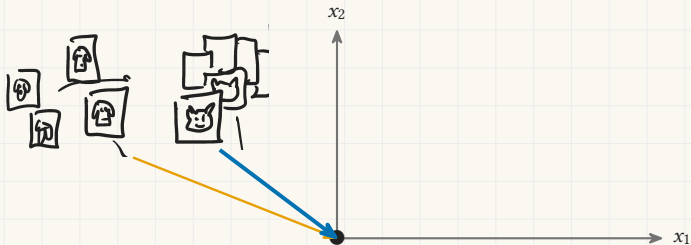
Diffusion models



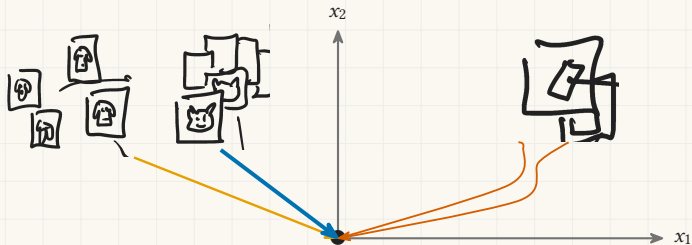
Diffusion models



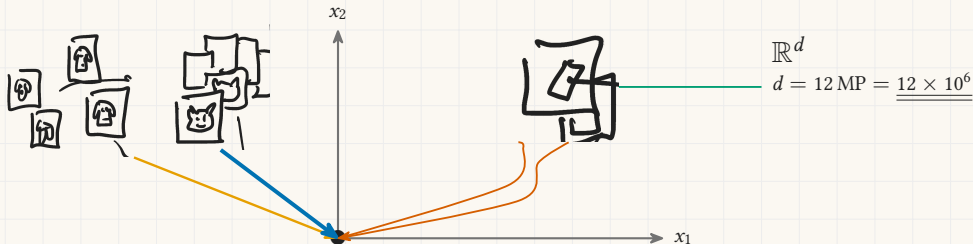
Diffusion models



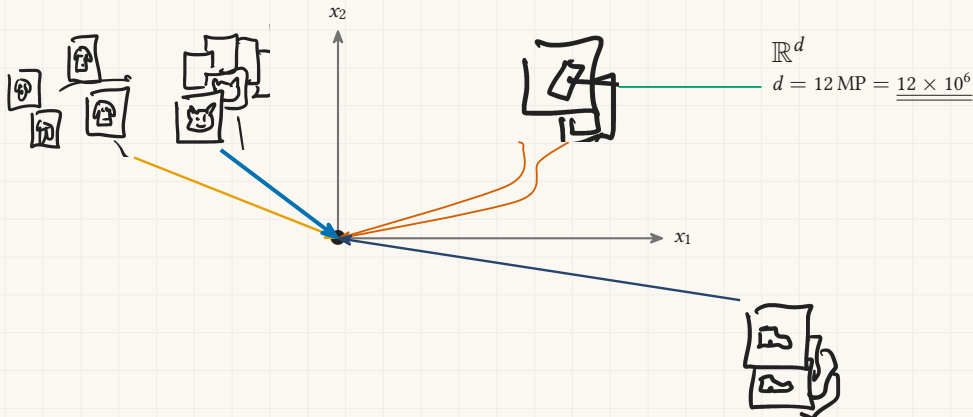
Diffusion models



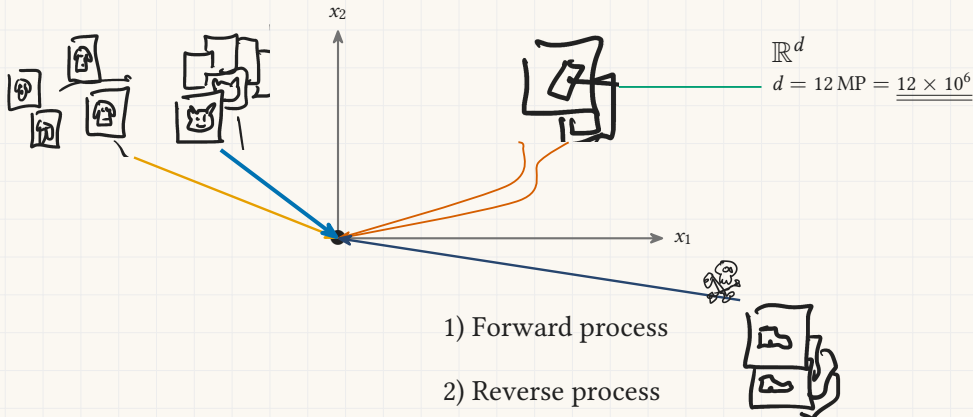
Diffusion models



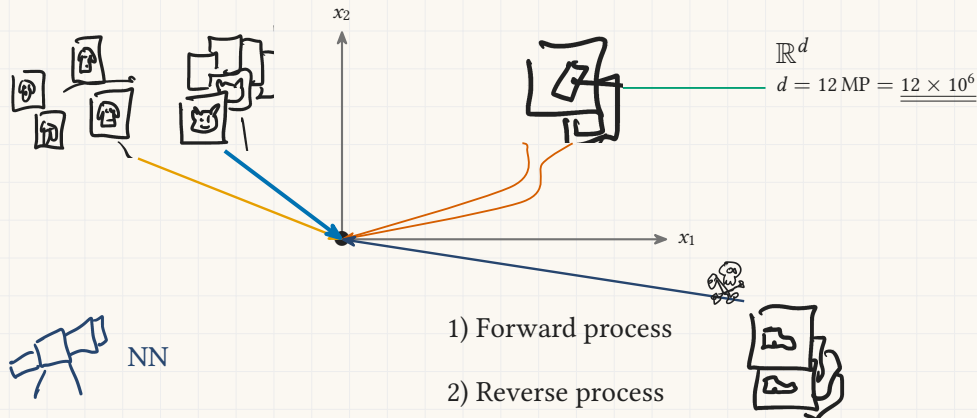
Diffusion models



Diffusion models



Diffusion models

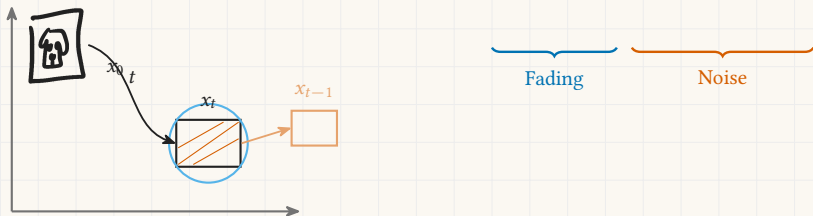


3

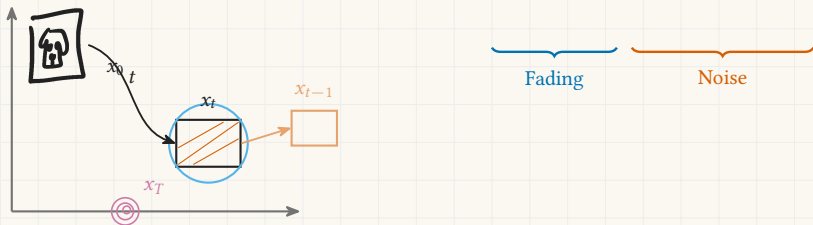
Section 3

Training and generating

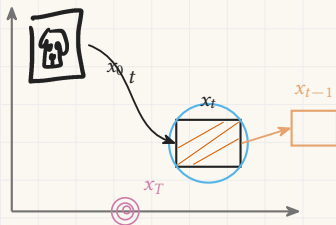
Training



Training

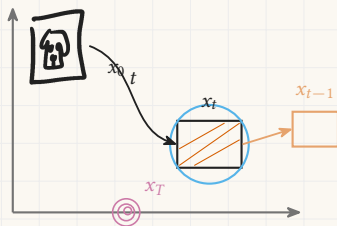


Training



$$x_t = \underbrace{\sqrt{\bar{\alpha}_t} x_0}_{\text{Fading}} + \underbrace{\sqrt{1 - \bar{\alpha}_t} \epsilon_t}_{\text{Noise}}$$

Training

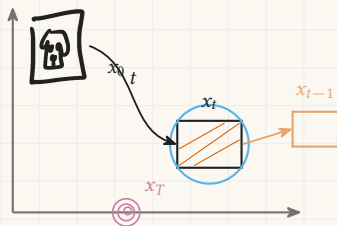


$$x_t = \underbrace{\sqrt{\bar{\alpha}_t} x_0}_{\text{Fading}} + \underbrace{\sqrt{1 - \bar{\alpha}_t} \epsilon_t}_{\text{Noise}}$$

$$\underbrace{x_t}_{\text{Fresh noise}} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_t \right) + \sqrt{\beta_t} z_t$$

Fresh noise

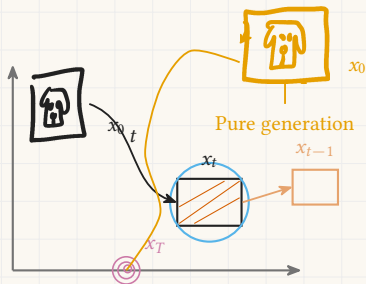
Training



$$x_t = \underbrace{\sqrt{\bar{\alpha}_t} x_0}_{\text{Fading}} + \underbrace{\sqrt{1 - \bar{\alpha}_t} \epsilon_t}_{\text{Noise}}$$

$$\underbrace{x_t}_{\text{Fading}} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_t \right) + \underbrace{\sqrt{\beta_t} z_t}_{\text{Fresh noise}}$$

Training

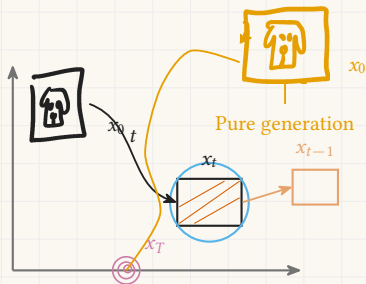


$$x_t = \underbrace{\sqrt{\bar{\alpha}_t} x_0}_{\text{Fading}} + \underbrace{\sqrt{1 - \bar{\alpha}_t} \epsilon_t}_{\text{Noise}}$$

$$\underbrace{x_t}_{\text{Fresh noise}} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_t \right) + \sqrt{\beta_t} z_t$$

Labels in the equations:
 - Fading: $\sqrt{\bar{\alpha}_t} x_0$
 - Noise: $\sqrt{1 - \bar{\alpha}_t} \epsilon_t$
 - Fresh noise: $\sqrt{\beta_t} z_t$

Training



$$x_t = \underbrace{\sqrt{\bar{\alpha}_t} x_0}_{\text{Fading}} + \underbrace{\sqrt{1 - \bar{\alpha}_t} \epsilon_t}_{\text{Noise}}$$

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_t \right) + \underbrace{\sqrt{\beta_t} z_t}_{\text{Fresh noise}}$$

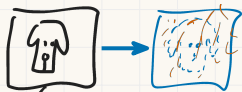


Problem: when generating out of thin air, I do not have

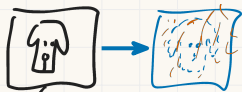
$\epsilon_0, \epsilon_1, \epsilon_2, \dots, \epsilon_t, \dots, \epsilon_T$

Solution: let a NN guess what the right ϵ_t is.

Training (in Vapnik's terms)



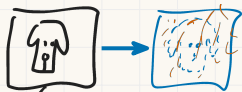
Training (in Vapnik's terms)



(Vapnik's)

Collect data $\{x^0, x^1, x^2, \dots, x^n\}$

Training (in Vapnik's terms)



(Vapnik's)

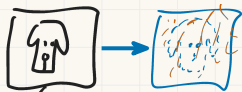
Collect data $\{x^0, x^1, x^2, \dots, x^n\}$

Model: $\text{NN}(x_t, t)$ tries to guess ε_t .

Training (in Vapnik's terms)



How:
do



(Vapnik's)

Collect data $\{x^0, x^1, x^2, \dots, x^n\}$

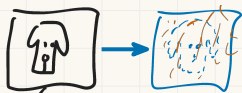
Model: $\text{NN}(x_t, t)$ tries to guess ε_t .

Training (in Vapnik's terms)



How:
do

Pick one image x^j

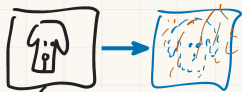


(Vapnik's)

Collect data $\{x^0, x^1, x^2, \dots, x^n\}$

Model: $\text{NN}(x_t, t)$ tries to guess ε_t .

Training (in Vapnik's terms)



How:
do

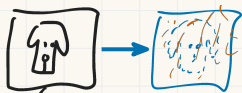
Pick one image x^j
Pick one $t \in \{0, \dots, T\}$

(Vapnik's)

Collect data $\{x^0, x^1, x^2, \dots, x^n\}$

Model: $\text{NN}(x_t, t)$ tries to guess ε_t .

Training (in Vapnik's terms)



How:
do

Pick one image x^j

Pick one $t \in \{0, \dots, T\}$

Compute x_t^j from x^j

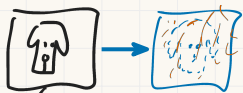


(Vapnik's)

Collect data $\{x^0, x^1, x^2, \dots, x^n\}$

Model: $\text{NN}(x_t, t)$ tries to guess ε_t .

Training (in Vapnik's terms)



(Vapnik's)

Collect data $\{x^0, x^1, x^2, \dots, x^n\}$

Model: $\text{NN}(x_t, t)$ tries to guess ε_t .

How:
do

Pick one image x^j

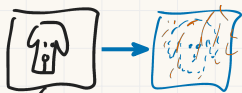
Pick one $t \in \{0, \dots, T\}$

Compute x_t^j from x^j

Track ε_t



Training (in Vapnik's terms)



(Vapnik's)

Collect data $\{x^0, x^1, x^2, \dots, x^n\}$

Model: $\text{NN}(x_t, t)$ tries to guess ε_t .

How:
do

Pick one image x^j

Pick one $t \in \{0, \dots, T\}$

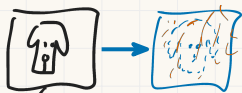
Compute x_t^j from x^j

Track ε_t

$$\hat{\varepsilon}_t = \text{NN}(x_t^j, t)$$



Training (in Vapnik's terms)



(Vapnik's)

Collect data $\{x^0, x^1, x^2, \dots, x^n\}$

Model: $\text{NN}(x_t, t)$ tries to guess ε_t .

How:
do

Pick one image x^j

Pick one $t \in \{0, \dots, T\}$

Compute x_t^j from x^j

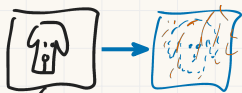
Track ε_t

$\hat{\varepsilon}_t = \text{NN}(x_t^j, t)$

$l = \sum_{ij} (\hat{\varepsilon}_{tij} - \varepsilon_{tij})^2$



Training (in Vapnik's terms)



(Vapnik's)

Collect data $\{x^0, x^1, x^2, \dots, x^n\}$

Model: $\text{NN}(x_t, t)$ tries to guess ε_t .

How:
do

Pick one image x^j

Pick one $t \in \{0, \dots, T\}$

Compute x_t^j from x^j

Track ε_t

$\hat{\varepsilon}_t = \text{NN}(x_t^j, t)$

$l = \sum_{ij} (\hat{\varepsilon}_{tij} - \varepsilon_{tij})^2$

$\hat{w}_{NN} \leftarrow \hat{w}_{NN} - \eta \nabla_{\hat{w}} l$

Until a fixed number of epochs

Generating

Produce x_T from white noise.

Generating

Produce x_T from white noise.

For $t = T, T - 1, T - 2, \dots$

Generating

Produce x_T from white noise.

For $t = T, T - 1, T - 2, \dots$

$$\hat{\varepsilon}_t = \text{NN}(x_t, t)$$

Generating

Produce x_T from white noise.

For $t = T, T - 1, T - 2, \dots$

$$\hat{\varepsilon}_t = \text{NN}(x_t, t)$$

Reverse x_{t-1} from x_t and $\hat{\varepsilon}_t$

Generating

Produce x_T from white noise.

For $t = T, T - 1, T - 2, \dots$

$$\hat{\varepsilon}_t = \text{NN}(x_t, t)$$

Reverse x_{t-1} from x_t and $\hat{\varepsilon}_t$

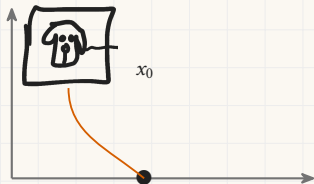
Print x_0

4

Section 4

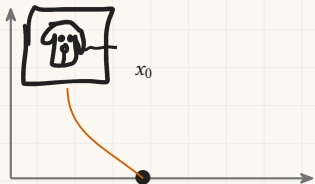
The forward and reverse formulas

Forward process



$\underbrace{\hspace{1.5cm}}$ $\underbrace{\hspace{1.5cm}}$
Fading Noise

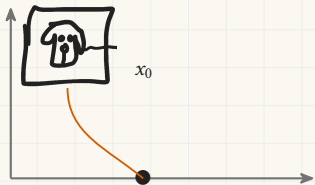
Forward process



Pick $\beta_t \in [0, 1]$

$$x_{t+1} = \underbrace{\sqrt{1 - \beta_t}}_{\text{Fading}} x_t + \underbrace{\sqrt{\beta_t} \varepsilon_t}_{\text{Noise}}$$

Forward process

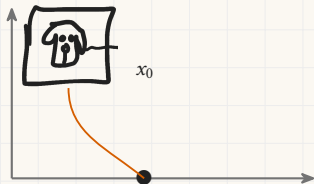


Pick $\beta_t \in [0, 1]$

$$x_{t+1} = \underbrace{\sqrt{1 - \beta_t}}_{\text{Fading}} x_t + \underbrace{\sqrt{\beta_t} \varepsilon_t}_{\text{Noise}}$$

Use the random generator from before: $\mu = 0$,
 $\sigma = 1$.

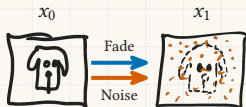
Forward process



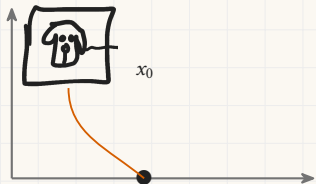
Pick $\beta_t \in [0, 1]$

$$x_{t+1} = \underbrace{\sqrt{1 - \beta_t}}_{\text{Fading}} x_t + \underbrace{\sqrt{\beta_t} \varepsilon_t}_{\text{Noise}}$$

Use the random generator from before: $\mu = 0$, $\sigma = 1$.



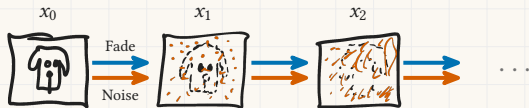
Forward process



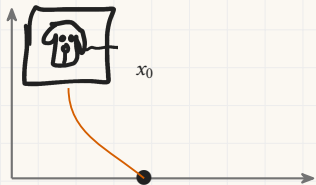
Pick $\beta_t \in [0, 1]$

$$x_{t+1} = \underbrace{\sqrt{1 - \beta_t}}_{\text{Fading}} x_t + \underbrace{\sqrt{\beta_t} \varepsilon_t}_{\text{Noise}}$$

Use the random generator from before: $\mu = 0$,
 $\sigma = 1$.



Forward process



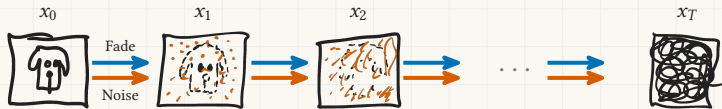
Pick $\beta_t \in [0, 1]$

$$x_{t+1} = \underbrace{\sqrt{1 - \beta_t}}_{\text{Fading}} x_t + \underbrace{\sqrt{\beta_t} \varepsilon_t}_{\text{Noise}}$$

Fading

Noise

Use the random generator from before: $\mu = 0$,
 $\sigma = 1$.



Forward

One-step formula

$$x_{t+1} = \sqrt{1 - \beta_t} x_t + \sqrt{\beta_t} \epsilon_t$$

Forward

One-step formula

$$x_{t+1} = \sqrt{1 - \beta_t} x_t + \sqrt{\beta_t} \epsilon_t$$
$$\sqrt{1 - \beta_{t-1}} x_{t-1} + \sqrt{\beta_{t-1}} \epsilon_{t-1}$$

Forward

One-step formula

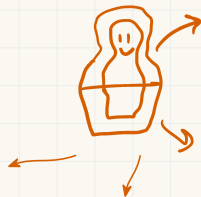
$$x_{t+1} = \sqrt{1 - \beta_t} x_t + \sqrt{\beta_t} \epsilon_t$$

$$\sqrt{1 - \beta_{t-1}} x_{t-1} + \sqrt{\beta_{t-1}} \epsilon_{t-1}$$

$$\sqrt{\dots} \dots \dots$$

⋮

x_0



Forward

One-step formula

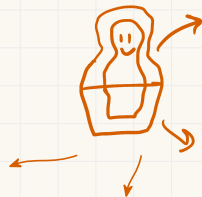
$$x_{t+1} = \sqrt{1 - \beta_t} x_t + \sqrt{\beta_t} \epsilon_t$$

$$\sqrt{1 - \beta_{t-1}} x_{t-1} + \sqrt{\beta_{t-1}} \epsilon_{t-1}$$

$$\sqrt{\dots} \dots \dots$$

$\vdots$

x_0



Fast-forward formula

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \hat{\epsilon}_t$$

Forward

One-step formula

$$x_{t+1} = \sqrt{1 - \beta_t} x_t + \sqrt{\beta_t} \epsilon_t$$

$$\sqrt{1 - \beta_{t-1}} x_{t-1} + \sqrt{\beta_{t-1}} \epsilon_{t-1}$$

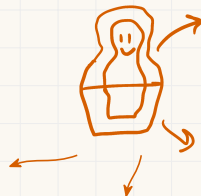
$$\sqrt{\dots} \dots \dots$$

$\vdots$

x_0

Fast-forward formula

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \hat{\epsilon}_t$$



One execution of the random procedure.

Forward

One-step formula

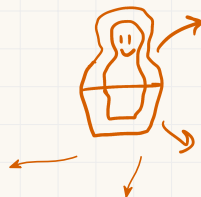
$$x_{t+1} = \sqrt{1 - \beta_t} x_t + \sqrt{\beta_t} \epsilon_t$$

$$\sqrt{1 - \beta_{t-1}} x_{t-1} + \sqrt{\beta_{t-1}} \epsilon_{t-1}$$

$$\sqrt{\dots} \dots \dots$$

$\vdots$

x_0



Fast-forward formula

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \hat{\epsilon}_t$$

One execution of the random procedure.

$$\alpha_t = 1 - \beta_t$$

Forward

One-step formula

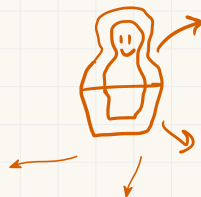
$$x_{t+1} = \sqrt{1 - \beta_t} x_t + \sqrt{\beta_t} \epsilon_t$$

$$\sqrt{1 - \beta_{t-1}} x_{t-1} + \sqrt{\beta_{t-1}} \epsilon_{t-1}$$

$$\sqrt{\dots} \dots \dots$$

$\vdots$

x_0



Fast-forward formula

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \hat{\epsilon}_t$$

One execution of the random procedure.

$$\alpha_t = 1 - \beta_t$$

$$\bar{\alpha}_t = \alpha_0 \alpha_1 \alpha_2 \cdots \alpha_t = \prod_{i \leq t} \alpha_i$$

Reverse

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_t \right) + \sqrt{\beta_t} z$$

$$\varepsilon_t$$

Reverse

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_t \right) + \sqrt{\beta_t} z$$

Black one $\rightarrow$ ε_t

Reverse

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_t \right) + \sqrt{\beta_t} z$$

Black one



ε_t



Total noise, added from x_0
till x_t

Reverse

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_t \right) + \sqrt{\beta_t} z$$

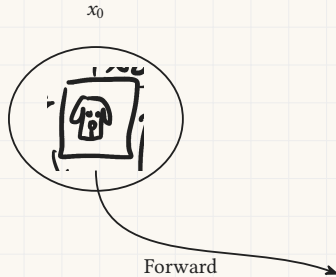
← Fresh noise

Black one

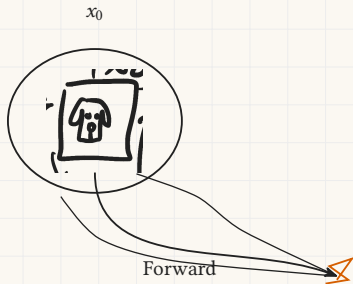
ε_t

Total noise, added from x_0
till x_t

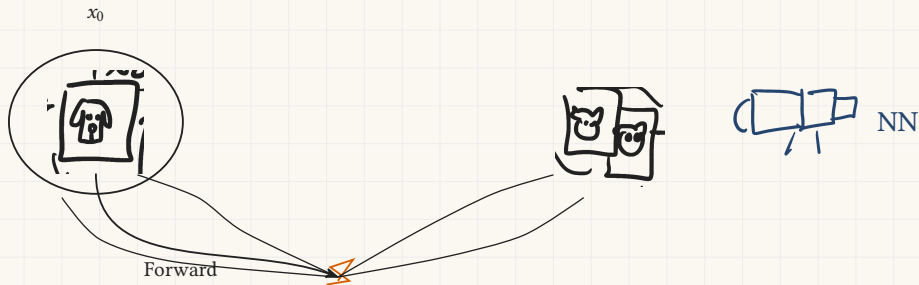
How to reverse?



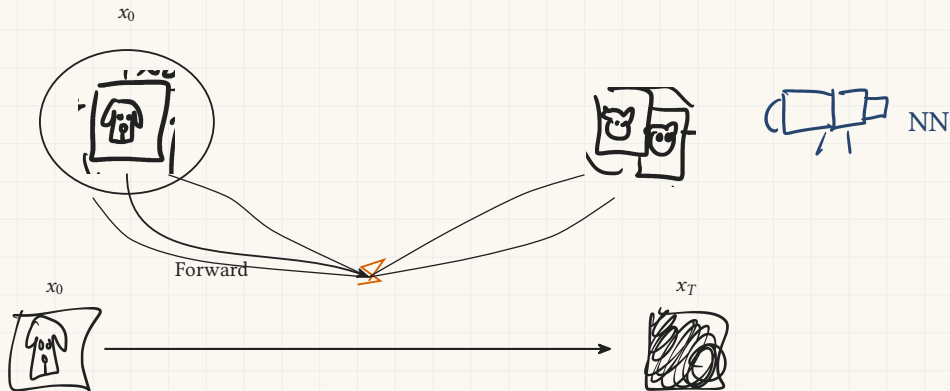
How to reverse?



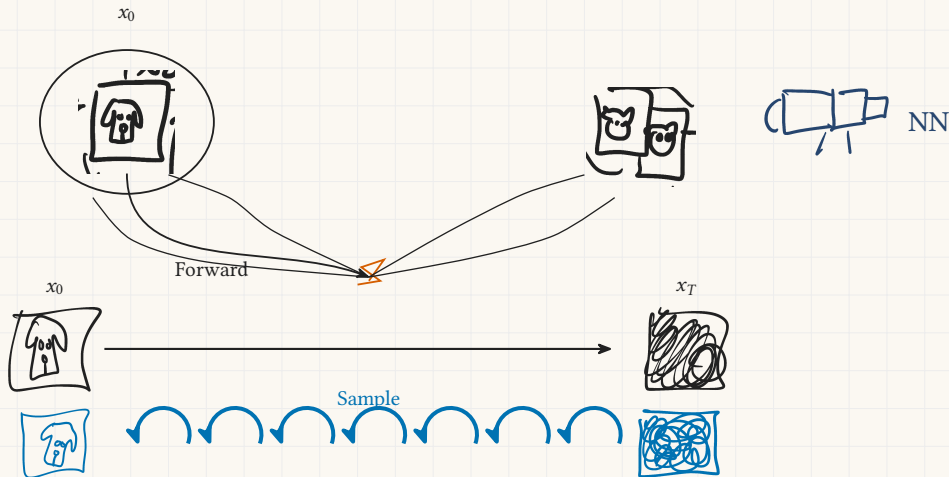
How to reverse?



How to reverse?



How to reverse?



How to guess the noise for a fresh image

- 1) Pick x from your dataset.

How to guess the noise for a fresh image

- 1) Pick x from your dataset.
- 2) Choose $t \in \{1, \dots, T\}$ at random.

How to guess the noise for a fresh image

- 1) Pick x from your dataset.
- 2) Choose $t \in \{1, \dots, T\}$ at random.
- 3) Forward $x_0, x_1, x_2, \dots, x_t$
while bookkeeping $\varepsilon_0, \varepsilon_1, \varepsilon_2, \dots, \varepsilon_t$

How to guess the noise for a fresh image

- 1) Pick x from your dataset.
- 2) Choose $t \in \{1, \dots, T\}$ at random.
- 3) Forward $x_0, x_1, x_2, \dots, x_t$
while bookkeeping $\varepsilon_0, \varepsilon_1, \varepsilon_2, \dots, \varepsilon_t$
- 4) Have the network predict ε_t from just x_t and t .

How to guess the noise for a fresh image

- 1) Pick x from your dataset.
- 2) Choose $t \in \{1, \dots, T\}$ at random.
- 3) Forward $x_0, x_1, x_2, \dots, x_t$
while bookkeeping $\varepsilon_0, \varepsilon_1, \varepsilon_2, \dots, \varepsilon_t$
- 4) Have the network predict ε_t from just x_t and t .

Then compare $\hat{\varepsilon}_t$ against ε_t .

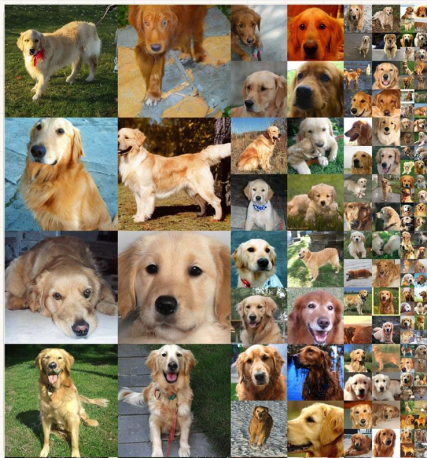
Prediction of the network

5

Section 5

Denoising diffusion probabilistic models

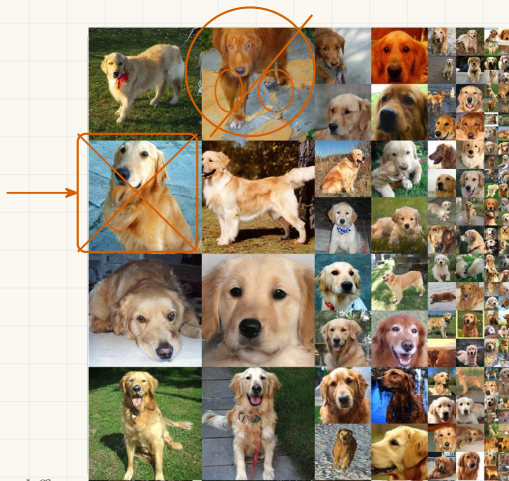
Which are real? Which are fake?



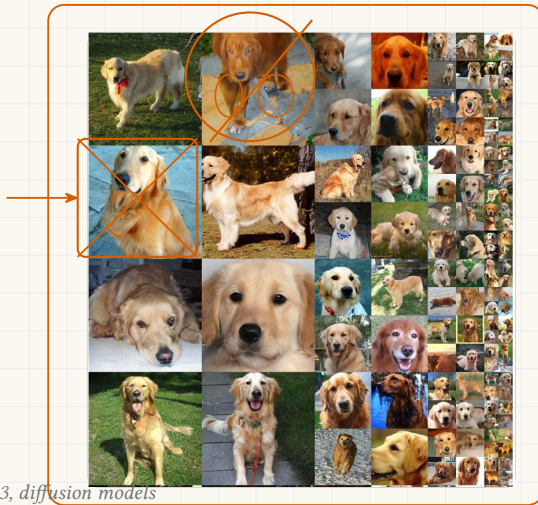
Which are real? Which are fake?



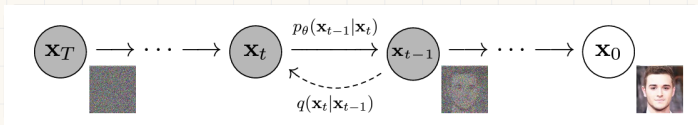
Which are real? Which are fake?



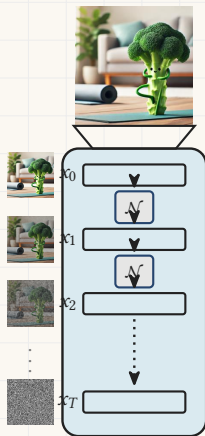
Which are real? Which are fake?



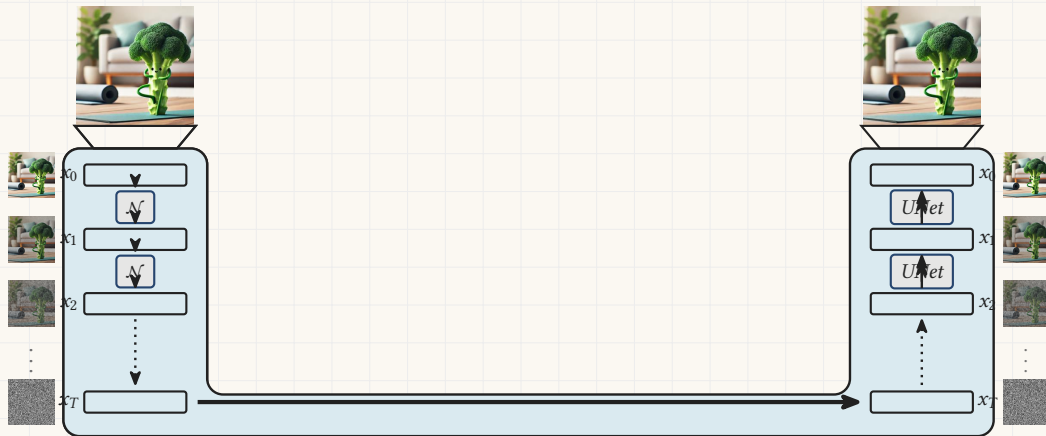
Denoising diffusion probabilistic models (DDPM), Ho et al., 2020



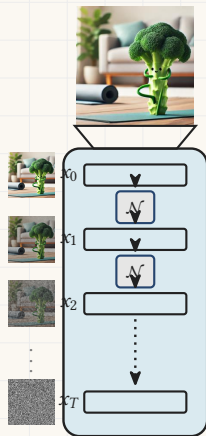
The forward and reverse chain



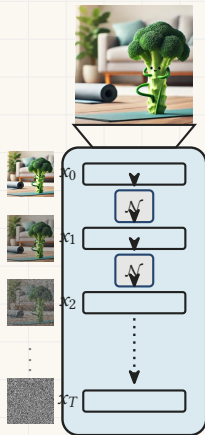
The forward and reverse chain



Forward process



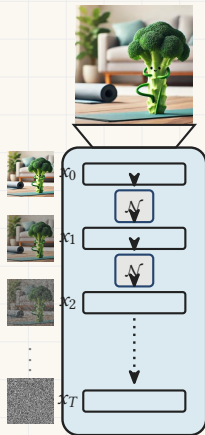
Forward process



For $t \leq T$, choose $\beta_t > 0$, let $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s \leq t} \alpha_s$. Then,

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t \mid \sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$$

Forward process



For $t \leq T$, choose $\beta_t > 0$, let $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s \leq t} \alpha_s$. Then,

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t | \sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$$

$$q(x_t | x_0) = \mathcal{N}(x_t | \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I)$$

Reverse process

For each $t \leq T$, the UNet tries to reconstruct **all noise** $\epsilon_\theta(x_t, t)$ that was added to form x_t from x_0 .

Reverse process

For each $t \leq T$, the UNet tries to reconstruct **all noise** $\epsilon_{\theta}(x_t, t)$ that was added to form x_t from x_0 .

We then try to reconstruct x_{t-1} as

follows

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right) + \sigma_t z$$

Reverse process

For each $t \leq T$, the UNet tries to reconstruct **all noise** $\epsilon_\theta(x_t, t)$ that was added to form x_t from x_0 .

We then try to reconstruct x_{t-1} as

follows

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z$$

Where σ_t is a chosen hyperparameter and z is sampled from a standard Gaussian.

Reverse process

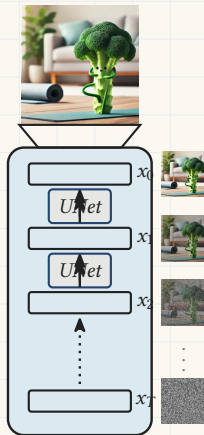
For each $t \leq T$, the UNet tries to reconstruct **all noise** $\epsilon_{\theta}(x_t, t)$ that was added to form x_t from x_0 .

We then try to reconstruct x_{t-1} as

follows

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right) + \sigma_t z$$

Where σ_t is a chosen hyperparameter and z is sampled from a standard Gaussian.

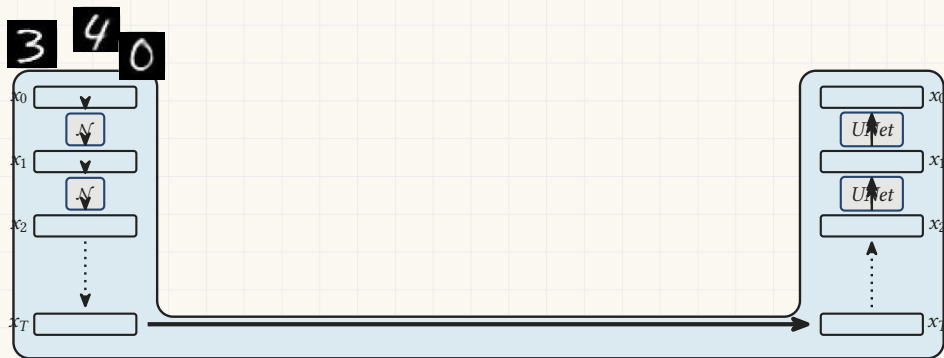


6

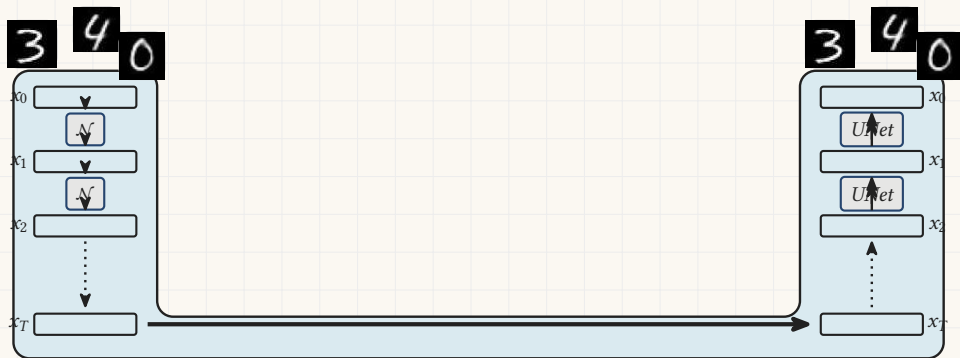
Section 6

Conditional diffusion processes

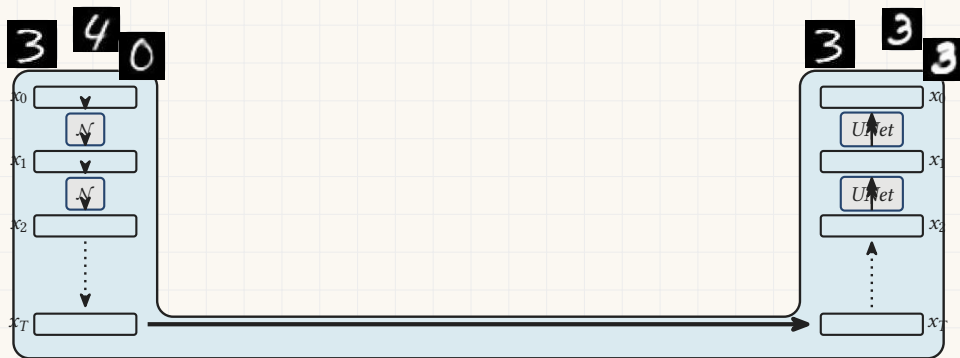
Diffusion process for MNIST



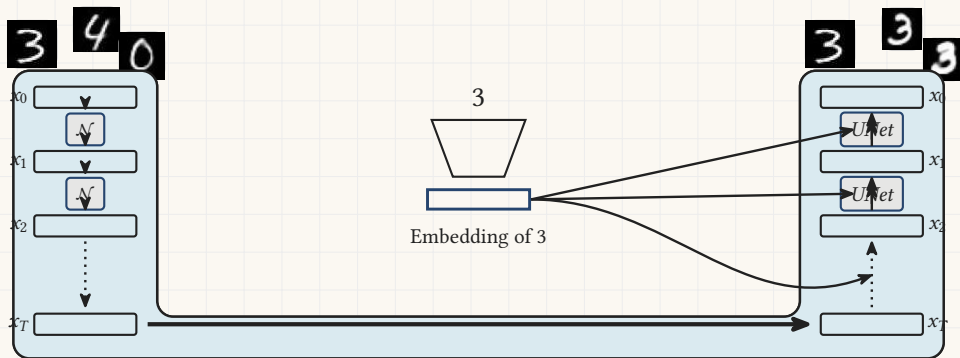
Diffusion process for MNIST



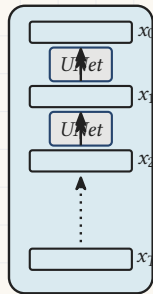
Conditional diffusion process for MNIST



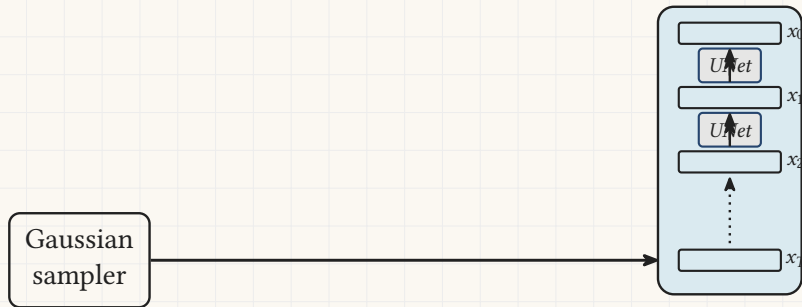
Conditional diffusion process for MNIST



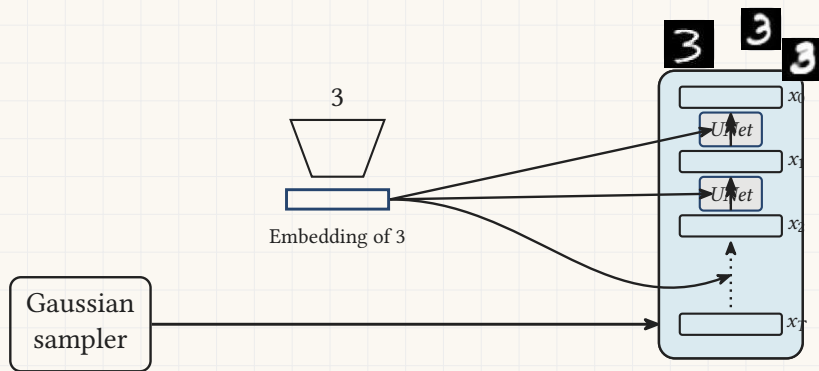
Conditional generation of handwritten digits



Conditional generation of handwritten digits



Conditional generation of handwritten digits



Link to an implementation with Hugging Face

<https://huggingface.co/learn/diffusion-course/en/unit2/3>

